Time to Intercept — l'interception orbitale

Starpwn CTF 2026 · DEF CON 34 · Aerospace Village

Catégorie : Space Operations **MÉTHODE COMPLÈTE — 27 août 2026**

Intercepter un satellite, mode d'emploi

Un satellite agresseur menace une infrastructure critique. Pour l'intercepter, il faut manœuvrer depuis une orbite circulaire basse jusqu'à l'orbite de la cible et **arriver au bon endroit au bon moment**. Le challenge n'est pas un fichier ni un site web : c'est un service réseau qui pose un scénario chiffré (altitude de départ, altitude de la cible, angle de phase actuel) et attend deux nombres en retour — l'impulsion à donner et le temps à attendre avant de la donner.

Concept — Un host:port n'est pas forcément un site web. Beaucoup de services CTF sont des **sockets TCP** bruts : des programmes qui attendent une connexion réseau directe et dialoguent en texte, sans page web. On ne les ouvre pas dans un navigateur : on s'y connecte avec `nc host port` (netcat, le « couteau suisse » des connexions réseau) ou un petit script Python. Le serveur envoie son message, on calcule, on renvoie la réponse sur la même connexion. Ici, il faut d'abord déployer l'instance sur la plateforme pour obtenir l'adresse et le port.

Pourquoi « MÉTHODE COMPLÈTE » et pas « RÉSOLU », honnêtement. Au moment de la rédaction, l'instance du service a expiré en fin de compétition et n'a pas pu être redéployée pour la capture finale : le solveur est validé sur le cas de référence LEO->GEO (voir plus bas) et la logique de dialogue avec le service a été testée pendant le CTF, mais la soumission définitive n'a pas pu être rejouée pour ce write-up. Plutôt que de maquiller le statut, autant le dire : la méthode est complète et vérifiée, la preuve de soumission date de la compétition.

Changer d'orbite au meilleur prix (le transfert de Hohmann)

Trois intuitions avant les formules

- **Une orbite, c'est tomber en permanence à côté de la Terre.** Un satellite avance si vite vers l'avant que, pendant qu'il tombe, la surface de la Terre s'écarte sous lui. Plus il est bas, plus la gravité le tient fort, et plus il doit aller vite pour ne pas redescendre : une orbite à 300 km d'altitude file à ~7,7 km/s, l'orbite géostationnaire à ~3 km/s.
- **LEO et GEO.** LEO (Low Earth Orbit) : l'orbite basse, quelques centaines de km — là où volent l'ISS et la plupart des satellites. GEO (Geostationary Earth Orbit) : 35 786 km, l'altitude magique où le satellite fait le tour de la Terre en exactement 24 h et paraît donc immobile au-dessus d'un point fixe — là où vivent les satellites TV et météo.
- **Le delta-v est la monnaie de l'espace.** On ne « monte » pas d'une orbite à l'autre en poussant en continu comme une voiture : on donne des **impulsions** brèves, et la mécanique céleste fait le reste. La quantité qui compte est le delta-v (la variation de vitesse, en m/s) : c'est elle qui coûte du carburant.

Le bond du sauteur (le transfert lui-même)

Pour passer d'une orbite circulaire basse à une orbite circulaire haute, la manœuvre la moins coûteuse en delta-v est le **transfert de Hohmann** (1925 — bien avant les fusées). L'idée : au lieu de viser la cible, on donne une impulsion qui place le vaisseau sur une **ellipse** dont le point le plus bas (périgée) touche l'orbite de départ et le point le plus haut (apogée) touche l'orbite d'arrivée. Comme un sauteur qui, d'un bond, passerait d'un trottoir roulant lent à un trottoir roulant rapide : le bond est l'impulsion, la trajectoire en cloche est l'ellipse.

On parcourt ensuite une **demi-ellipse** — exactement 180°, du périgée à l'apogée — ce qui prend une demi-période de cette ellipse. À l'arrivée, une seconde impulsion circulariserait l'orbite, mais le challenge ne demande que la première : l'interception compte plus que le stationnement.

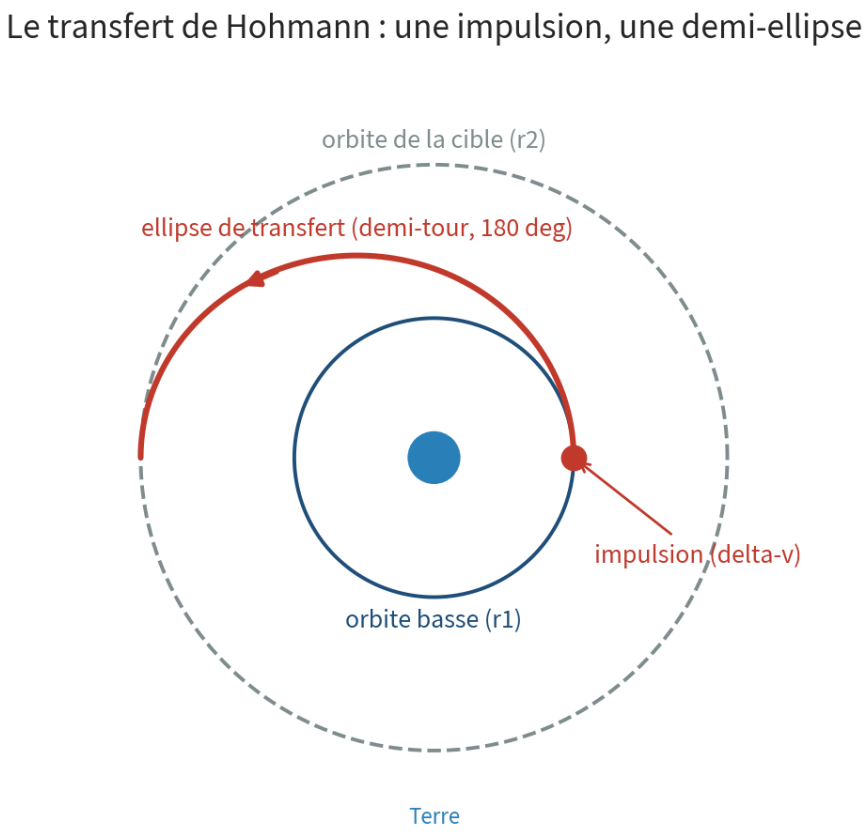


Figure 1 — Le transfert de Hohmann : la premiere impulsion, sur l orbite basse, injecte le vaisseau sur l ellipse de transfert (rouge) ; une demi-revolution plus tard, il atteint l apogee, sur l orbite de la cible.

Le « quand » qui tue (le phasage)

Calculer le delta-v ne suffit pas : arriver sur la bonne orbite alors que la cible est **de l'autre côté de la Terre** ne sert à rien. Tout le sel du challenge est là. Il faut tirer à l'instant précis où la cible a, devant nous, l'avance angulaire exacte qui la fera arriver au point de rendez-vous en même temps que nous.

Concept — Pourquoi il faut attendre. Le point de rendez-vous est à l'apogée, soit à 180° de la position de tir. Pendant la durée du transfert t_t, la cible (vitesse angulaire w2) avance sur son orbite d'un angle w2·t_t. Pour qu'elle soit au rendez-vous quand le vaisseau y arrive, elle doit se trouver, au moment du tir, à un angle **theta = pi - w2·t_t** devant. C'est comme doubler un coureur sur un anneau : si le sprint dure 20 secondes et que le coureur parcourt 30 mètres en 20

secondes, il faut démarrer le sprint quand il a exactement 30 mètres d'avance sur le point d'arrivée — pas avant, pas après.

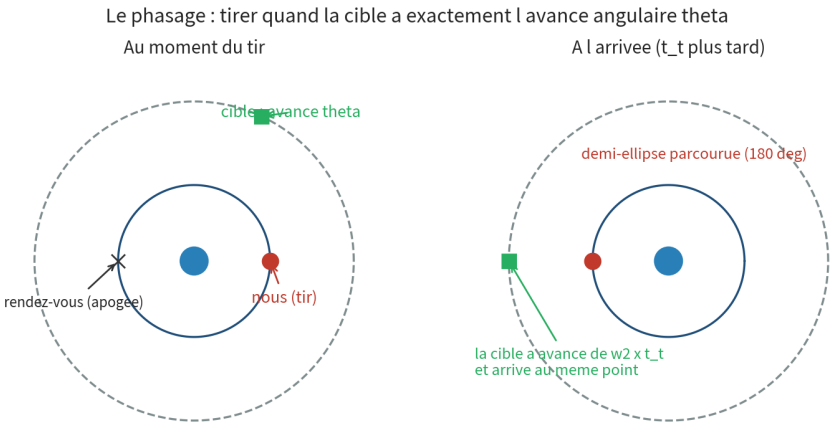


Figure 2 — Le phasage : au moment du tir (gauche), la cible doit avoir l avance angulaire theta ; pendant la demi-ellipse (180 deg), elle avance sur son orbite et rejoint le rendez-vous (droite).

Or l'orbite basse tourne plus vite que l'orbite haute (w1 > w2, cf. l'intuition ci-dessus) : l'angle de phase entre les deux évolue au rythme w1 - w2. Il suffit donc d'**attendre** que cet angle atteigne la valeur theta requise. Ce temps d'attente est le second nombre à renvoyer. Sur un anneau, c'est le moment où le poursuivant plus rapide voit son retard fondre jusqu'à la valeur exacte du départ de sprint.

Les formules (promis, elles sont gentilles)

Avec mu = 398 600,44 km3/s2 (le **paramètre gravitationnel** de la Terre — la constante qui résume à elle seule « à quel point la Terre attire »), R = 6 371 km (rayon terrestre), r1 = R + h1 (rayon orbital de départ) et r2 = R + h2 (celui de la cible) :

```
a   = (r1 + r2) / 2                          demi-grand axe de l'ellipse
dv  = sqrt(mu*(2/r1 - 1/a)) - sqrt(mu/r1)    1re impulsion (km/s -&gt; *1000 = m/s)
t_t = pi * sqrt(a**3 / mu)                   duree du transfert (demi-periode)
w1  = sqrt(mu / r1**3)   (toi, rapide)       vitesses angulaires des
w2  = sqrt(mu / r2**3)   (cible, lente)      deux orbites circulaires
theta_requis = (pi - w2 * t_t) mod 2pi       avance que la cible doit avoir
phi0         = angle de phase actuel (rad)   (lu dans le scenario)
t_attente    = ((phi0 - theta_requis) mod 2pi) / (w1 - w2)
```

Lecture : `a` est la « taille » de l'ellipse de transfert ; `dv` compare la vitesse qu'il faut sur l'ellipse au périgée à la vitesse qu'on a sur l'orbite basse — la différence est l'impulsion à donner ; `t_t` applique la troisième loi de Kepler (la période ne dépend que de la taille de l'ellipse) divisée par deux ; le reste est la gestion de la phase vue plus haut. La réponse à envoyer est la paire `delta_v_burn wait_time` (tolérances typiques ±10 m/s et ±60 s, plusieurs essais par connexion).

Le crash-test avant le vrai vol (LEO -> GEO)

Avant de faire confiance au script, on le teste sur un cas dont la réponse est **tabulée partout** : le transfert d'une orbite basse (300 km) vers l'orbite géostationnaire (35 786 km) — la manœuvre que font les satellites TV après leur lancement. La valeur de référence de la première impulsion est d'environ 2 440 m/s.

```
LEO 300 km -&gt; GEO 35786 km :
```

```
1re impulsion calculee : 2427.7 m/s   (reference ~2440 m/s)  OK
duree de transfert     : 18982 s  = 5,27 h  (demi-orbite GTO)  OK
```

L'écart de ~12 m/s tient au choix du rayon terrestre (6 371 vs 6 378 km) et reste dans la tolérance. La méthode est donc validée avant même de se connecter au vrai service. C'est la même discipline que dans les autres write-ups de cette série : un **témoin** dont on connaît la réponse, qui sépare les erreurs de formule des erreurs de dialogue avec le service.

50 minutes pour un signe « moins »

Une seule subtilité pratique a résisté, et elle a coûté près d'une heure : le **signe de l'angle de phase** donné par le serveur. Selon sa convention (angle mesuré cible-devant-nous ou nous-devant-la-cible), le temps d'attente sort cohérent mais faux — et un temps d'attente faux ne se voit pas dans les formules : tous les nombres sont plausibles, toutes les unités sont bonnes, il se voit uniquement dans le refus du serveur. Cinquante minutes à vérifier le delta-v (juste), la demi-période (juste), les unités (justes), avant de soupçonner la convention. La solution n'est pas élégante mais elle est efficace : le script tente theta puis -theta automatiquement, dans la limite des essais autorisés par connexion. Le témoin LEO->GEO avait prouvé que la physique était bonne ; c'est ce qui a permis de circonscrire la panne à la convention de signe au lieu de tout reprendre.

La morale de l'histoire

- **Une adresse host:port n'est pas forcément un site web** : beaucoup de services CTF sont des sockets TCP, à contacter avec netcat ou un script, après déploiement de l'instance.
- **Le calcul orbital, c'est deux problèmes, pas un** : le transfert de Hohmann donne le « combien » (le delta-v) ; le phasage donne le « quand » (le temps d'attente). Oublier le second fait rater la cible même avec le bon delta-v.
- **Valider sur un cas connu avant le cas réel** : vérifier le solveur sur LEO->GEO (2 428 m/s attendus) garantit qu'une réponse refusée vient d'une convention de signe, pas d'une formule fausse — c'est ce qui a borné la recherche de panne à 50 minutes au lieu d'une soirée.

Méthode et script validés sur le cas de référence LEO->GEO ; les deux conventions de signe sont tentées automatiquement. Le solveur : `github.com/ShymusQuantum/starpwn-2026` (`time_to_intercept/hohmann_solver.py`).